



# An agent, a world with no instructions, and a way to find out what actually helps.

ARC-AGI-3 is a set of small games: a 64×64 screen, sixteen colours, a few buttons, and no rules given. The agent has to work out what the buttons do and what winning means — by acting.

Five capabilities were added to an agent to help it do that. None of them beat random play. That is the honest starting point, and it is the reason this is worth joining rather than a finished story.

## WHAT HAS BEEN TRIED SO FAR

|    |                                                                  |                                 |
|----|------------------------------------------------------------------|---------------------------------|
| E0 | A frozen open-weight model picks every move from the screen      | NOT ABOVE RANDOM                |
| E1 | Harness remembers failures and blocks moves known to do nothing  | HELPED SEARCH, NOT SOLVING      |
| E2 | Harness tests each button first and tells the model what it does | WORSE — ABANDONED               |
| E3 | Model labels each move LEARN or WIN                              | WORSE — ABANDONED               |
| E4 | Random play that skips moves known to change nothing             | STRONG LOCALLY, DIDN'T TRANSFER |

Every one of these was written down *with its own falsifier* before it ran. That is why a shelf of negative results is a contribution instead of a dead end.

Pick the one that matches what you want out of it.

These are different competitions with different rules. The choice is not about how much you contribute — it changes what you are allowed to see.

#### TRACK A • PAPER

##### Run experiments, co-author the paper

For most people. You run your own experiment on your own laptop and write up what you find.

- No GPU, no API key, no cost
- You own one experiment end to end
- Co-author on the experiment you ran
- No deadline pressure to merge teams

#### TRACK B • COMPETITION

##### Join the Kaggle team

For people who want the leaderboard. Limited seats, and one hard date.

- Maximum 8 people on a team
- Must merge on Kaggle before **26 October**
- The whole team shares one submission per day
- Prize money splits evenly across the team

#### THE RULE EVERYONE HAS TO KNOW

To download the games you must accept the competition rules, which makes you an official competitor — even on the paper track.

Competitors on different teams **may not privately share code**. So the private research harness never changes hands. You work from the public agent and your own code. Nothing flows the wrong way, and nobody can breach anything by accident.

A laptop and about an hour.

The strongest policy found so far has no model in it at all. That is the thing to beat, and it runs on anything.

|                                   |                      |                                           |
|-----------------------------------|----------------------|-------------------------------------------|
| <b>0</b><br>GPUs needed           | <b>0</b><br>API keys | <b>&lt; 1 h</b><br>compute per experiment |
| <b>25</b><br>public games to play |                      |                                           |

Get the baseline running before you touch an idea.

If your setup is wrong, everything you measure afterwards is worthless. Better to find out in the first hour than in week three.

#### 1 Clone the process repo

```
git clone https://github.com/Goutham-Eda/arc-agi3-experiment.git
cd arc-agi3-experiment
```

#### 2 Install — four packages, nothing exotic

```
python -m venv .venv
source .venv/bin/activate          # Windows:
.venv\Scripts\Activate.ps1
pip install arc-agi arcengine numpy pytest
```

3

## Get the games yourself

Accept the competition rules on Kaggle, download the data, unzip it. Nobody can send these to you — the data is not redistributable. Then point at it:

```
export ARC_ENV_DIR=/path/to/environment_files
# PowerShell: $env:ARC_ENV_DIR =
"C:\path\to\environment_files"
```

4

## Run the gate

```
python run_local.py --games 3 --episodes 3 --max-actions
80    # ~1 min
python run_local.py --games 0 --episodes 4 --max-actions
300   # all 25 games
```

ok It finds the games

ok **Every episode starts at level 1.** If one starts higher, you are reusing a game world between tries — a bug that once inflated a baseline three-fold.

ok **No level cleared in under 5 moves.** The shortest genuine clear ever seen took five. Anything faster is a bug, not a breakthrough.

ok **Almost nothing gets cleared.** The reference run clears 9 levels in 100 episodes. A great score here means something is wrong, not right.

5

## Point your coding agent at it

Optional, but the repo ships rules that override an agent's instinct to run first and think later.

```
cp -r arc-agi3-experiment ~/.claude/skills/    # Claude
Code
# Codex and others read AGENTS.md in the repo root
```

## One experiment, start to finish. About a week.

The stages exist to separate two things that look identical from the outside: *my idea was wrong* and *my code was broken*.

### WRITE

Fill in the preregistration — twelve fields.

The two that matter: what number counts as success, and what result makes you give up. Both decided now, before any run.

### SMOKE

Run it on one game.

Does it finish without crashing? That is the only question. No conclusions.

### PROBE

Run on practice games, a couple of settings.

Is your program actually doing what you said? Is the new feature really switched on? This is the step that saves a wasted week.

### TUNE

Run properly on the 8 practice games.

Settings get fixed here — or your give-up rule fires here.

### ↩ STOP

If the give-up rule fires, **stop and write it up. This is a finished experiment**, not a failure. Three of the first five ended exactly here.

### HELD-OUT

Run once on the 17 games you have never touched.

This is the verdict. You do not peek at these games earlier — not once, not "just to check".

### WRITE-UP

Fill in the result form.

Include the safety checklist, the per-game table, and a direct answer to: did my idea fail, or did my code fail?

 **THREE RULES**

Everything else is detail.

---

**Write down what would prove you wrong, before you run.**

Decide what success looks like after seeing the numbers and you will always find something that qualifies. Writing it first is not paperwork — it is the whole method.

---

**Practice games can kill an idea. They can never prove one.**

Settings get chosen on the 8 practice games. Verdicts come only from the 17 held-out games. A promising practice number is not a result, however good it looks.

---

**Always report every game, not just the total.**

A doubled score here once turned out to come almost entirely from one level of one game. The total said "strong method". The per-game table said "one game". Only one of those was true.

---

 **CLAIM ONE**

Open questions, one owner each.

Each of these asks whether one specific missing capability is what random play lacks. They are independent, so nobody blocks anybody.

| ID | THE QUESTION, IN PLAIN WORDS                                                                                               | NEEDS       |
|----|----------------------------------------------------------------------------------------------------------------------------|-------------|
| P1 | Do we misjudge when the screen changed, because counters tick? — already run; the "clocks" turned out to be progress bars  | taken       |
| P2 | Should it click on <i>objects</i> it can see, instead of random spots on the screen?                                       | laptop      |
| P3 | Can it learn, <i>during a single game</i> , which buttons actually do something — and is that better than guessing evenly? | laptop      |
| P4 | If it remembers where it has been and deliberately walks back to unexplored places, does it get further?                   | laptop      |
| P5 | If it already <i>knew</i> the goal, how much better would it do? Sets the ceiling for goal-inference work.                 | pen & paper |

Why the simple ones matter: if a plain statistical policy on a laptop beats the expensive reasoning harness, then the expensive harness was never necessary. These are the controls that make the whole result mean something.

 [START HERE](#)

Everything you need is public.

#### The process repo

[github.com/Goutham-Eda/arc-agi3-experiment](https://github.com/Goutham-Eda/arc-agi3-experiment)

#### The public baseline agent

[kaggle.com/code/goutham12/arc-agi-3-build-3-random-prune](https://kaggle.com/code/goutham12/arc-agi-3-build-3-random-prune)

#### Competition — accept rules, get the games

[kaggle.com/competitions/arc-prize-2026-arc-agi-3](https://kaggle.com/competitions/arc-prize-2026-arc-agi-3)

#### Paper track — where the write-up goes

[kaggle.com/competitions/arc-prize-2026-paper-track](https://kaggle.com/competitions/arc-prize-2026-paper-track)

---

Read `SKILL.md` first, then `references/setup.md`. Everything in the repo is method and templates — no competition code, no competition data.

Key dates: team merger closes **26 October 2026** · final submission **2 November 2026**. The paper track is unaffected by both.